

Overdetermined Systems - Requirements and Restrictions

Practical requirements, experiment-preservation rules, and computing conditions for reproducing the dissertation benchmark

1. Purpose of this document

This document sets out the conditions I used when carrying out the large overdetermined-system experiments in the dissertation. It is intended for someone who wants to reproduce the work from the repository, including a reader who has not used this particular benchmark before. The purpose is not to prescribe one permanent computer configuration. The mathematical experiment needs to remain the same, while some computing settings have to be selected according to the current University of Manchester CSF rules and the resources available to the researcher.

The two current program files for this section are **Overdetermined.py** and **Overdetermined_plots.py**. The first is the numerical experiment. It constructs the test problems, runs the solver calculations and writes the numerical results. The second takes the resulting CSV data and performs the plotting and summary stage. The accompanying introduction explains what the experiment is measuring; the separate run-instructions document explains the practical execution steps.

2. The experiment is a set of parameter studies, not six fixed cases

One point is important before changing any settings. The overdetermined benchmark is not defined by six individual matrices. It is a parameterised experimental programme. Six study categories are used in the repository so that one property of the problem can be examined at a time. The program then runs the relevant parameter values and records the complete configuration with the results.

Study	Parameter varied	Values used in the dissertation
Matrix Size	m, with n fixed	n = 5000; m = 20000, 25000, 30000
Condition Parameter	kappa	10, 100, 1000, 10000, 20000
Regularisation / Lambda	lambda	0, 1e-8, 1e-6, 1e-4, 1e-2, 1e-1, 1
Sparsity	s	0.90, 0.95, 0.98, 0.99, 0.999
Sparsity Pattern	matrix structure	random, banded, block
Spectrum	spectral construction	linear, exponential, polynomial, clustered

The common large overdetermined setting is $n = 5000$ and, for most studies, $m = 20000$. The condition-parameter study is the exception: it uses $m = 51000$ and $n = 5000$, with sparsity 0.95, a random pattern, an exponential spectrum and $\lambda = 1$. The matrix-size study changes m itself. These details matter because changing them changes the numerical problem rather than merely changing an input label.

3. What must not be changed when reproducing the dissertation experiment

The safest way to reproduce the results is to use the supplied **Overdetermined.py** as the implementation of the benchmark rather than rewriting the numerical method in a different form. In particular, the following parts should be preserved:

- the matrix dimensions and parameter values used by each study;
- the matrix-generation procedure, including the selected sparsity pattern and spectral construction;
- the random-number generation and seed/repeat handling;
- the construction of the known solution and noisy right-hand side;
- the solver formulations and solver labels used by the program;

- the stopping criteria, tolerances and maximum-iteration settings;
- the regularisation formulation and the lambda values used in the study;
- the quantities recorded in the CSV output, including runtime, iterations, residual/error measures and convergence information.

The dissertation used Direct, CG, LSQR and LSMR for the synthetic large overdetermined comparison. The implementation should therefore not be replaced by a different algorithm simply because it has the same general purpose. In particular, the supplied matrix-free CG formulation should not be replaced by explicitly forming the normal-equation matrix if the aim is to reproduce the supplied implementation.

4. Python and software environment

The benchmark was developed using Anaconda 2024.10 together with the scientific Python packages used by the supplied programs, including NumPy, SciPy, pandas and Matplotlib. A compatible environment is required. If a different Python or package version is used, that difference should be recorded rather than hidden.

This matters particularly for a performance experiment. Different versions of numerical libraries can use different sparse-linear-algebra routines or BLAS/LAPACK backends. Hardware and processor architecture can also change runtime. Small differences in the last digits of a numerical result do not necessarily mean that the experiment is wrong, but changes to the actual mathematical configuration should not be made without recording them.

5. CSF and Slurm

The large overdetermined benchmark was run as a University of Manchester CSF workload using Slurm. The repository does not attempt to teach the University's CSF login procedure. Anyone reproducing the experiment needs to obtain the appropriate access and follow the University's current instructions for logging into CSF and using the available compute partitions.

The recorded workflow was organised as a 24-element Slurm array. This is a description of how the benchmark workload was organised, not a historical job number. A new submission will receive its own Slurm job identifier, and that identifier is not part of the experiment.

6. Memory requirement for the Direct solver

Memory is the most important resource constraint for the large Direct-solver runs. In the configuration used for this benchmark, allocations below **128 GB of RAM are expected to fail for the Direct-solver cases**. For that reason, at least 128 GB should be requested when reproducing those cases, subject to what the University currently permits.

The reason is not simply the size of the original sparse matrix. The Direct calculation forms and factorises a regularised normal-equation system. Factorisation can create intermediate storage and fill-in, so the peak memory requirement can be substantially greater than the storage needed for the input matrix alone.

The 128 GB figure should be treated as the practical requirement for this documented benchmark, not as a mathematical statement that every conceivable implementation of a direct method requires 128 GB. Before submitting a job, the researcher must check the current University CSF allocation rules and confirm that the required memory is available to the relevant account, project and partition.

7. Wall time and CPU allocation

The recorded benchmark workflow used a **24-hour wall-time allocation**. This should be treated as the reference arrangement for reproducing the recorded experiment. Current CSF limits take precedence: queue policies, partition limits and project allocations can change, so the current rules must be checked before submission.

CPU allocation is deliberately not fixed in this document. The number of CPUs available per job is dependent on the researcher, account, project and current CSF allocation. The person reproducing the experiment should check the University's current CPU and partition limits, choose an allowed CPU count, and record the value used.

CPU count, memory, wall time and software environment should be recorded alongside timing results. Runtime is a measurement of the whole computational setup, not just of the Python algorithm in isolation.

8. Slurm resource template

The following is a record of the important resource settings rather than a universal submission script. Current CSF account, partition and resource requirements must be checked before it is used.

```
#!/bin/bash
#SBATCH --job-name=overdetermined_benchmark
#SBATCH --time=24:00:00
#SBATCH --mem=128G
#SBATCH --cpus-per-task=<CPU_COUNT>
#SBATCH --array=1-24

# Activate the appropriate University-supported Python/Anaconda environment.
python Overdetermined.py
```

The placeholder **<CPU_COUNT>** must be replaced with a value that is actually available under the current CSF allocation. An account or partition directive may also be required. Those values are intentionally left out because they depend on the current University configuration.

9. Running the benchmark and producing the derived files

The normal workflow is divided into two stages. The numerical program is run first. Only after its CSV results have been produced should the plotting program be run. Keeping these stages separate means that figures can be recreated later from the numerical results without repeating the expensive large-system calculations.

Stage 1 - Numerical experiment

- Place Overdetermined.py in the working directory on the CSF system.
- Load or activate the compatible Python/Anaconda environment.
- Submit the numerical program through the appropriate Slurm arrangement.
- Allow the benchmark to complete and check the Slurm output for errors.
- Locate the CSV result files written by Overdetermined.py and retain them for the plotting stage.

Stage 2 - Plotting and summaries

- Place Overdetermined_plots.py where it can access the numerical CSV results.
- Run the plotting program in the compatible Python environment.
- The plotting program reads the CSV data, groups results by the relevant study parameter and solver, and produces the figures and summary CSV outputs defined by the program.
- Check the output directory and confirm that the expected plots and summary files have been created.

The exact output filenames should be taken from the current supplied plotting program rather than guessed from an older execution. This is especially important if the code is updated in the future.

10. Why the CSV and PNG files do not have to remain in the repository

The CSV and PNG files produced by the benchmark are derived outputs. They are not required as source inputs for the numerical experiment itself. The repository can therefore be kept relatively clean by retaining the Python programs and documentation while removing generated outputs after they have been inspected or archived separately.

For an archived dissertation run, it is still useful to keep the generated CSV and PNG files separately with the repository version, software versions, CSF resource information and execution date. That provides an audit trail for the particular run without making generated data a permanent dependency of the reproducibility repository.

11. Randomness and numerical reproducibility

The random-number configuration is part of the experiment. The matrix, the true solution and the noise are generated in a controlled way, and the dissertation describes independent random-number streams for these components with corresponding repeats using the same seed arrangement. This should not be changed when trying to reproduce the reported results.

The right-hand side is constructed from a known solution with controlled noise. The dissertation uses a relative noise level of 0.01. Consequently, the numerical results depend on more than the matrix dimensions: the random matrix realisation, true solution, noise and repeat handling all contribute.

12. What can be adapted to the current CSF system

Some parts of the original computing environment cannot sensibly be frozen in a repository. The following may need to be changed to comply with current University requirements:

- the CSF login procedure and account setup;
- the partition or queue used;
- the account or project directives required by CSF;
- the number of CPUs available to the job;
- the exact memory allocation syntax or permitted memory limits;
- the queue limits and maximum permitted wall time;
- the activation command for the current supported Python/Anaconda environment.

These are environmental adaptations. They are different from changing the experiment. For example, changing a Slurm directive because the current CSF system requires a different syntax is reasonable; changing a matrix dimension or solver tolerance simply to make a job finish is not a reproduction of the same benchmark.

13. Common failure points

If a large Direct run fails, the first checks should be the requested memory and the Slurm error output. A low-memory failure should not be interpreted as evidence that the mathematical benchmark is invalid. The practical requirement for the Direct cases is at least 128 GB, provided that allocation is available under the current University rules.

If an iterative solver takes substantially longer than expected, check the CPU allocation, software environment, current system load and the actual parameter configuration. A different CPU allocation can change runtime even when the Python source is unchanged.

If the plotting stage fails, first check that the numerical CSV files exist and that their location and column structure match what `Overdetermined_plots.py` expects. The plotting stage cannot recreate missing numerical results by itself.

14. Information to record for a reproducible run

When reproducing or extending the benchmark, the following information should be recorded with the results. This makes it possible to tell whether a difference is caused by the mathematical experiment or simply by the computing environment.

- repository version or Git commit used;
- execution date;
- Python version and relevant package versions;
- CSF partition or queue;
- memory requested and, where available, memory actually used;
- CPU count requested and relevant hardware information where available;
- wall-time limit;
- Slurm array structure used;
- any changes made to the software or execution environment;
- locations of the generated numerical CSV files and plotting outputs.

Timing values should always be interpreted with these details. A runtime measured with one CPU allocation, memory allocation, processor architecture and software stack is not automatically comparable with a runtime obtained under a different configuration.

15. Reproduction checklist

- The current University procedure for CSF access has been followed.
- A compatible Python/Anaconda environment has been checked.
- The current repository versions of `Overdetermined.py` and `Overdetermined_plots.py` are being used.
- The correct parameter study and parameter values have been selected.
- The matrix-generation, random-number, solver, tolerance and repeat settings have not been silently changed.
- At least 128 GB of RAM is requested for the Direct-solver cases, where that allocation is available and permitted.
- The current CSF rules have been checked for memory, CPU, partition and wall-time restrictions.
- The recorded workflow's 24-hour wall-time arrangement is used where current CSF rules allow it.
- The CPU count used for the job is recorded.
- The 24-element array structure is preserved where it is required for the corresponding benchmark workflow.
- The numerical stage is completed before the plotting stage.
- Generated CSV and PNG files are treated as derived outputs and may be archived separately from the source repository.

16. Final point

For this section, reproducibility means keeping the mathematical experiment and the supplied implementation intact while adapting only the computing settings that genuinely depend on the current CSF environment. The repository contains the code and documentation needed to reconstruct the experiment; the researcher supplies the current University access, resource allocation and execution environment. Recording both sides of that arrangement is what makes the resulting timings and numerical results meaningful to compare.